Deep Learning: Autoencoders y Autoencoders Variacionales

TP5: Autoencoder y Denoising Autoencoder sobre letras; VAE sobre sprites de Pokémon

Britez, Santiago Kantor, Marcos Sánchez Toledo, Mariano

Sistemas de Inteligencia Artificial — TP5

Instituto Tecnológico de Buenos Aires

2026

Dos ejercicios, una pregunta: ¿qué hace útil a un espacio latente?

Ej. 1 — Autoencoder con letras (7×5)

Representar 32 letras con **2 números reales** y recuperarlas con ≤ 1 píxel mal; después, quitar ruido (*denoising*).

Ej. 2 — VAE con sprites de Pokémon (32×32 RGB)

El autoencoder reconstruye, pero su espacio latente tiene **huecos**. El término KL del VAE lo vuelve **muestreable** y permite generar.

0 px

de error en las 32 letras con el modelo final (un PCA con 2 componentes deja hasta 15 px)

6,8 vs. 19,1

distancia de Fréchet de las muestras del prior al conjunto: VAE vs. autoencoder (d=32, 3 semillas). Piso con Pokémon reales no vistos: 4,1

Objetivos: (1) qué arquitectura y optimizador permiten comprimir; (2) qué aporta entrenar con ruido; (3) qué cambia el esquema variacional; (4) generar una muestra nueva y justificar que pertenece al conjunto.

Todo el aprendizaje está escrito desde cero en `numpy` y validado

Implementación propia

- Capas densas y retropropagación a mano.
- Optimizadores SGD, Momentum y Adam.
- Autoencoder, Denoising AE y VAE (reparametrización, KL cerrada, factor β).
- Métricas: error en píxeles y distancia de Fréchet.

Bibliotecas de soporte (no resuelven el TP)

`numpy`: álgebra matricial (incluida la SVD). `PIL`: leer y escalar imágenes. `matplotlib`, `pandas`: gráficos y tablas.

$$\sim 10^{-8}$$

error relativo del gradiente analítico frente a diferencias finitas

(AE: $8{,}8 \cdot 10^{-9}$; VAE: $1{,}8 \cdot 10^{-8}$)

Semillas fijas y varias por configuración (5 en el Ej. 1, 3 en el Ej. 2); hiperparámetros en `config.json`.

Clave

El objetivo es **entender qué controla la representación**: arquitectura, optimizador, ruido y término variacional.

Ej. 1a: 32 letras de 35 píxeles deben caber en un plano de 2 dimensiones



font.h de la cátedra: 32 patrones binarios de 7×5 (0x60–0x7f).

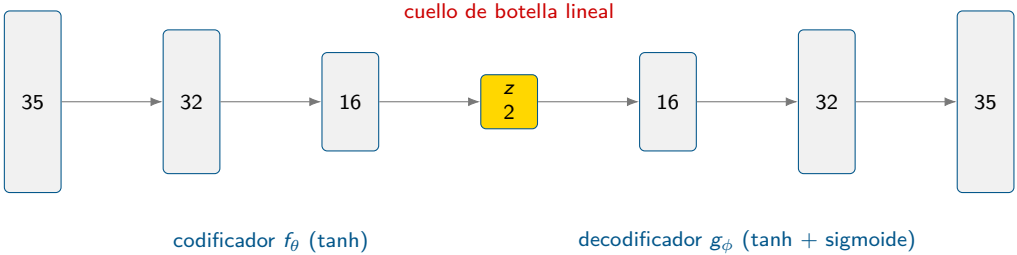
Información vs. optimización

32 patrones requieren $\log_2 32 = 5$ bits: dos reales alcanzan de sobra. La dificultad es que el decodificador **dibuje 32 islas** y que el descenso por gradiente las encuentre.

Criterio de éxito del enunciado

Peor patrón con ≤ 1 píxel incorrecto tras umbralizar $\hat{x} > 0{,}5$.

Arquitectura: cuello de botella lineal de 2 unidades y entropía cruzada



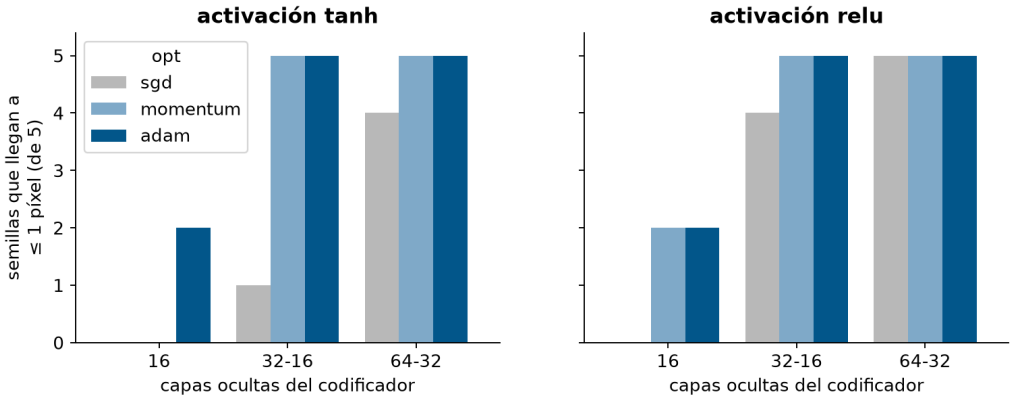
Pérdida

$$\mathcal{L} = -\sum_i \left[x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i) \right]$$

Por qué entropía cruzada binaria

Los píxeles son 0/1. El gradiente respecto del logit, $\partial \mathcal{L} / \partial a_i = \hat{x}_i - x_i$, no lleva el factor $\sigma'(a)$: la red sigue aprendiendo aunque se equivoque con confianza (con error cuadrático + sigmoide se estanca).

Una capa oculta no alcanza; con dos, Adam llega en las 5 semillas



5 semillas por barra; 3000 épocas *full-batch*; SGD (η=0,03), Momentum (0,005), Adam (0,005). Misma escala en ambos paneles. Con dos capas ocultas Adam necesita una mediana de 200–500 épocas; Momentum, 2 a 5 veces más.

El modelo final recupera las 32 letras sin errores; el PCA lineal no puede



0 px

peor patrón, autoencoder

($\leq$ 1 px desde la época 450)

15 px

peor patrón, PCA 2 comp.

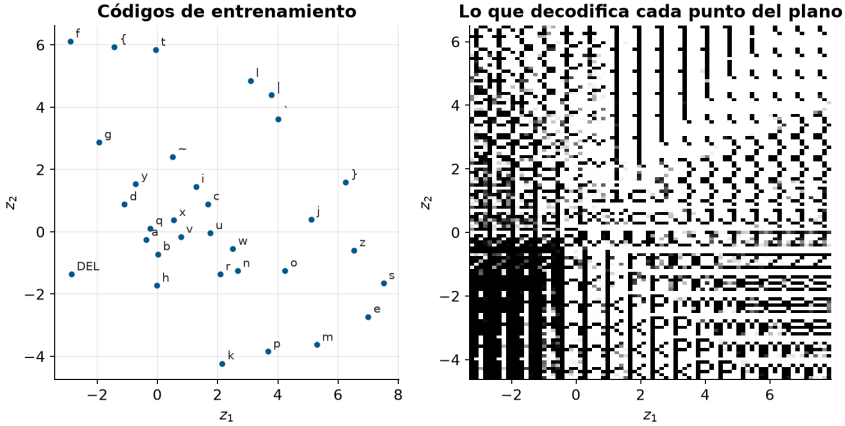
(7,2 en promedio)

35–32–16–2 `tanh`, Adam
$\eta = 5 \cdot 10^{-3}$, 6000 épocas, semilla 0.
La no linealidad es lo que permite que 2 dimensiones alcancen.

Clave

Un autoencoder lineal converge a PCA (teorema visto en clase; Baldi & Hornik, 1989): lo que aporta el modelo no lineal es una **variedad curva** en lugar de un plano.

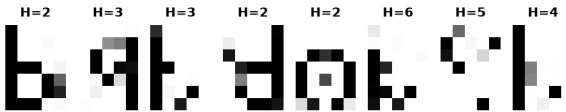
El espacio latente 2-D ubica las 32 letras, pero entre ellas quedan huecos



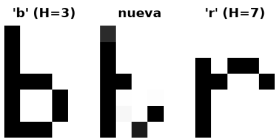
Izquierda: los 32 códigos (entre $-4{,}2$ y $7{,}5$). Derecha: lo que decodifica cada punto de una grilla del plano. Cerca de un código se ve su letra; entre códigos aparecen mezclas e imágenes sin sentido. **Nada en la pérdida ordena este espacio.**

Decodificar un punto intermedio genera una letra nueva: una mezcla de sus vecinas

candidatas (H = Hamming a la letra más cercana)



elegida, entre sus dos vecinas



Cómo se elige

Grilla de 41×41 puntos en el rango de los códigos. Se retienen las imágenes (i) a distancia de Hamming ≥ 2 de **toda** letra de entrenamiento y (ii) nítidas (≤ 2 píxeles grises): 171 de 1681.

Qué significa "nueva"

La elegida ($z = (0{,}35;\ -0{,}73)$) está a 3 píxeles de la "b" y a 7 de la "r". No es una letra del alfabeto: es una **interpolación**. Que "parezca una letra" es un juicio humano.

Ej. 1b: se entrena con la entrada corrupta y se exige reconstruir la limpia

Diseño del experimento

- Ruido: cada píxel se invierte con probabilidad p (sal y pimienta sobre imágenes binarias).
- Entrada ruidosa $\tilde{x}$, **objetivo limpio** x: $\mathcal{L}\big(x,\, g_\phi(f_\theta(\tilde{x}))\big)$.
- Ruido **nuevo en cada época**: con 32 patrones, un ruido fijo se memorizaría.
- Misma arquitectura del 1a (35–32–16–d), $d \in \{2, 8\}$, $p_{train} \in \{0;\ 0{,}05;\ 0{,}1;\ 0{,}2\}$, 5 semillas.

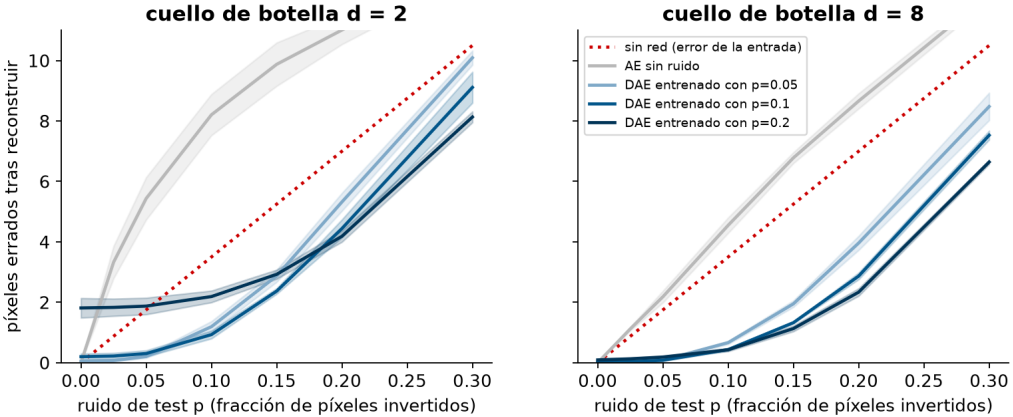
Por qué la misma arquitectura

Para aislar el efecto del ruido. $d = 8$ permite comparar con un cuello de botella menos estrecho.

0,9 → 3,3 px

con 2,5 % de ruido, el autoencoder entrenado sin ruido (d=2) *agrega* errores

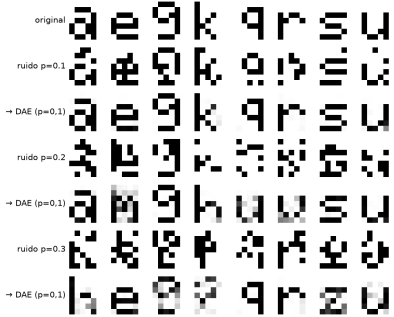
Con $d = 8$ el DAE deja 0,44 px donde la entrada tenía 3,5



Media $\pm$ desvío entre 5 semillas; 32 letras $\times$ 20 muestras de ruido por punto; semillas de ruido de evaluación distintas de las de entrenamiento.

Misma escala vertical. Línea roja punteada: error de la entrada sin red (35 p).

El DAE devuelve letras válidas; con demasiado ruido, la letra equivocada

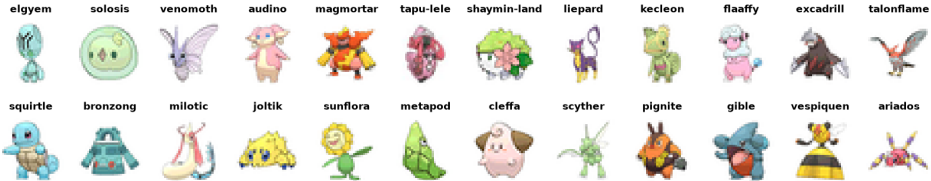


DAE d=8 entrenado con p=0,1 (semilla 0). Con p=0,3 la entrada queda más cerca de otra letra: error *confiado*, análogo a los estados espurios de Hopfield (TP4).

Clave

El ruido de entrenamiento es un compromiso: protege a niveles altos pero cuesta precisión sin ruido (d=2, p_{train}=0,2: 1,8 px aun con la entrada limpia; con d=8, 0,09).

Ej. 2: un conjunto nuevo y más exigente, 809 sprites de Pokémon



Por qué este conjunto

- **Sin clases**: cada Pokémon aparece una vez; no hay prototipos que memorizar.
- **En color**: 3 canales, el modelo también aprende paletas.
- Criterio claro de éxito al generar: que no sea ninguno de los 809 y que igual parezca un Pokémon.

Preprocesamiento y partición

- Composición sobre fondo blanco, recorte a la figura, centrado con 6 % de margen.
- Escalado a 32×32 RGB: 3072 valores en $[0, 1]$.
- **81 Pokémon completos (10 %) retenidos**; los 728 restantes se duplican con su reflejo horizontal (1456 imágenes).

La red base es la del Ej. 1 a otra escala; cada decisión se tomó midiendo

Arquitectura e hiperparámetros

- Codificador 3072–512–256–d; decodificador simétrico con salida sigmoide.
- **ReLU en lugar de tanh**: con 3072 entradas tanh satura (prueba preliminar: BCE de entrenamiento 888 vs. 701 nats).
- Adam, $\eta = 3 \cdot 10^{-4}$, lotes de 64, 300 épocas. Con $\eta = 10^{-3}$ sobreajusta.
- $d \in \{2, 32\}$: con 2 el espacio se puede **ver**; con 32 se puede **dibujar**.

BCE con píxeles continuos

Su mínimo no es 0 sino la entropía de los propios píxeles: se compara contra un piso y un techo.

600 – 981

nats por imagen: reconstrucción perfecta vs. predecir siempre la imagen media (Pokémon no vistos)

Clave

El autoencoder del Ej. 1 no dice **dónde** quedan los códigos: para generar no sabemos de qué distribución sacar z. Eso es lo que resuelve el esquema variacional.

VAE: el codificador devuelve una distribución y la pérdida suma un regularizador

Modelo generativo e inferencia variacional

$z \sim p(z) = \mathcal{N}(0, I)$, $\quad x \mid z \sim p_\phi(x \mid z)$ (Bernoulli por píxel).

$p(z \mid x)$ es intratable: se aproxima con $q_\theta(z \mid x) = \mathcal{N}\big(\mu_\theta(x), \operatorname{diag} \sigma^2_\theta(x)\big)$.

$$\log p(x) = \mathrm{KL}\big(q(z|x) \,\|\, p(z|x)\big) + \mathcal{L} \quad \Rightarrow \quad \mathcal{L} \le \log p(x)$$

$$-\mathcal{L} = \underbrace{-\mathbb{E}_q\big[\log p_\phi(x|z)\big]}_{\text{reconstrucción (BCE)}} + \underbrace{\mathrm{KL}\big(q_\theta(z|x) \,\|\, \mathcal{N}(0, I)\big)}_{\text{regularizador}}$$

Qué hace cada término

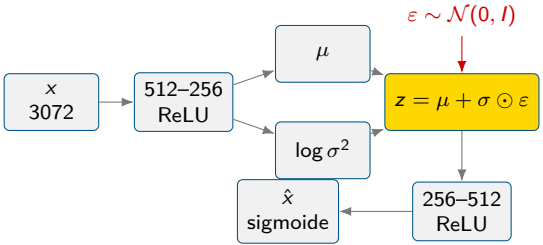
Reconstrucción: cada z muestreado debe decodificar en algo parecido a x.

KL: empuja cada $q(z|x)$ hacia el prior; los códigos cubren $\mathcal{N}(0, I)$ sin huecos.

Factor β (Higgins et al., 2017)

Pérdida $= \mathrm{BCE} + \beta \cdot \mathrm{KL}$. $\beta = 1$: cota variacional. $\beta = 0$: sin regularizador, se comporta como el autoencoder del Ej. 1.

La reparametrización permite retropropagar a través del muestreo



El azar entra por ε; μ y σ quedan en un camino derivable.

KL en forma cerrada (gaussianas diagonales)

$$\mathrm{KL} = \tfrac{1}{2} \sum_{j=1}^{d} \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right)$$

Gradientes que llegan al codificador

$$\partial \mathcal{L} / \partial \mu = \delta_z + \beta \mu$$

$$\partial \mathcal{L} / \partial \log \sigma^2 = \tfrac{1}{2}\, \delta_z \odot \varepsilon \odot \sigma + \tfrac{\beta}{2} (\sigma^2 - 1)$$

δ_z: error de reconstrucción retropropagado por el decodificador.

Clave

El decodificador sólo recibe gradiente de reconstrucción; el codificador, reconstrucción más KL. Verificado con diferencias finitas fijando ε.

¿Cómo medir que una muestra "pertenece" al conjunto si no hay etiquetas?

Distancia de Fréchet entre distribuciones

Se ajusta una gaussiana a cada nube de rasgos y se calcula

$$\|m_1 - m_2\|^2 + \mathrm{tr}\left(S_1 + S_2 - 2\,(S_1 S_2)^{1/2}\right)$$

Es la idea del FID (Heusel et al., 2017). Rasgos: las 20 primeras componentes principales del entrenamiento (SVD de `numpy`).

Un piso para interpretarla

La misma distancia para los **81 Pokémon reales no vistos**, con igual tamaño de muestra: **4,1**. Si las muestras generadas se acercan al piso, se parecen al conjunto tanto como un Pokémon real nuevo.

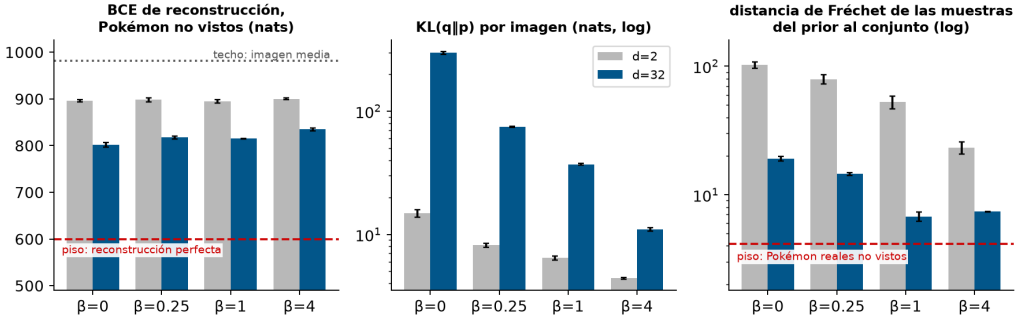
Fracción de casi copias

Muestras más cerca de un dato de entrenamiento que la mitad de la distancia típica de un Pokémon nuevo. Una copia no es una muestra nueva.

Clave

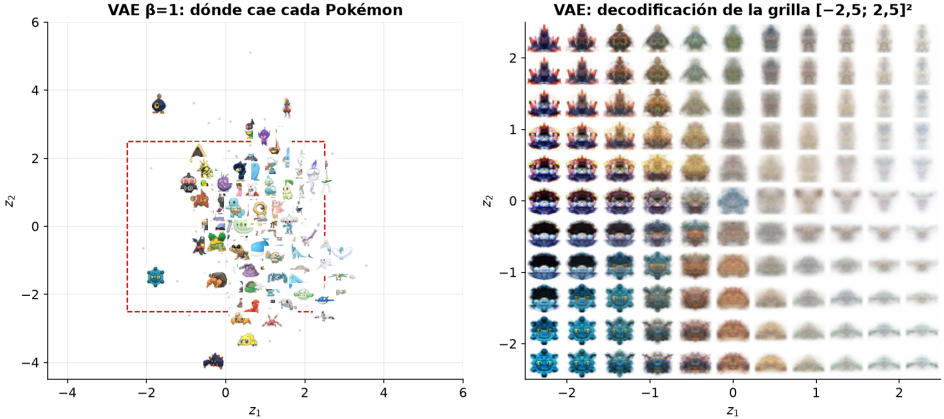
No se evalúa una imagen suelta sino la **distribución** de 2000 muestras de $z \sim \mathcal{N}(0, I)$, siempre contra una referencia de datos reales.

El KL cuesta algo de reconstrucción, pero acerca las muestras al conjunto real



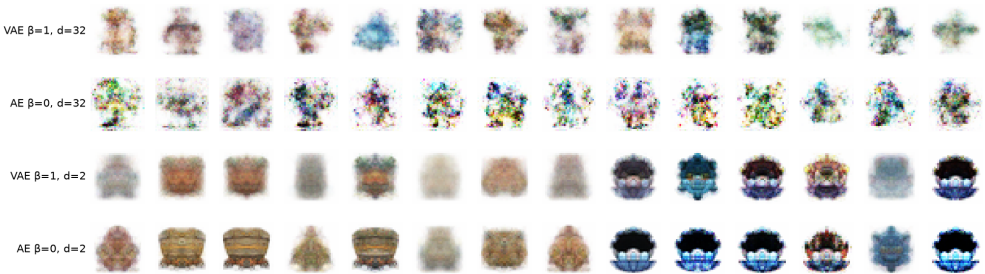
24 redes: $\beta \in \{0;\ 0{,}25;\ 1;\ 4\} \times d \in \{2;\ 32\} \times$ 3 semillas (barras de error = desvío). Con d=32: BCE 802 → 815 nats, KL 299 → 37 nats, Fréchet 19,1 → 6,8 al pasar de β=0 a β=1. β=4 no mejora las muestras (7,4) y eleva las casi copias de 1 % a 9 %.

En 2 dimensiones el VAE ordena el espacio por color y silueta, sin etiquetas



VAE β=1, d=2, semilla 0. Izquierda: media posterior μ de cada Pokémon de entrenamiento (cuadro rojo: $[-2{,}5;\ 2{,}5]^2$, donde cae el 92 % de los códigos; con β=0, el 74 %). Derecha: decodificación de la grilla. Transiciones suaves, pero imágenes borrosas: d=2 sirve para ver, no para dibujar.

Con las mismas $z \sim \mathcal{N}(0, I)$, el autoencoder da ruido y el VAE, figuras



7,4

Fréchet, VAE d=32
(semilla 0; piso 4,1)

18,6

Fréchet, AE d=32
(semilla 0)

Los códigos del AE tienen desvío $\approx$ 2,6 y 299 nats de KL: el prior cae en zonas que su decodificador nunca vio. Son los **huecos** del Ej. 1, ahora en 32 dimensiones.

Un Pokémon nuevo: no es copia y está tan cerca del conjunto como uno real



Arriba: muestra elegida y sus tres vecinos reales más cercanos (espacio PCA).

Abajo: 10 muestras filtradas y su vecino real.

Filtro sobre 2000 muestras del prior (d=32)

(1) no son casi copias; (2) área y contraste de la figura entre los percentiles 10 y 90 de los sprites reales. Quedan **629**.

Argumentos de pertenencia

- En píxeles: vecino a 6,1 (umbral de copia 4,9).
- En PCA: vecino a 2,82 (Pokémon real no visto, mediana 4,54).
- Las 629 filtradas: mediana 4,56 vs. 4,54 de los reales.

El juicio final es humano: se reconoce por silueta y paleta, no por detalles.

Ej. 2b: con convoluciones escritas a mano desaparece el ruido de colores



5,2

Fréchet, VAE conv.

(MLP 6,8; piso 4,1)

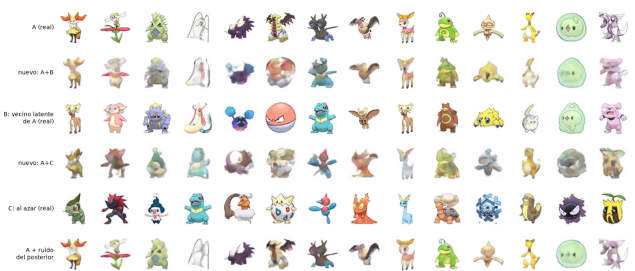
798

nats en no vistos

(MLP 815)

Mismos datos, pérdida, latente y métricas; sólo cambian las capas densas por **convoluciones**, con 7 veces menos parámetros. La retropropagación está escrita a mano (error de gradiente 10^{-8}); `cupy` hace las cuentas de arreglos en GPU con la interfaz de `numpy`. A 64×64 (4 h de GPU) las figuras son más limpias, pero **siguen sin ojos ni contornos**.

Ej. 2b: la figura más convincente es el punto medio entre dos Pokémon vecinos



Modelo convolucional de 64×64. $z = \frac{1}{2}(\mu_A + \mu_B)$; B es el vecino más cercano de A en el espacio latente y C un Pokémon al azar.

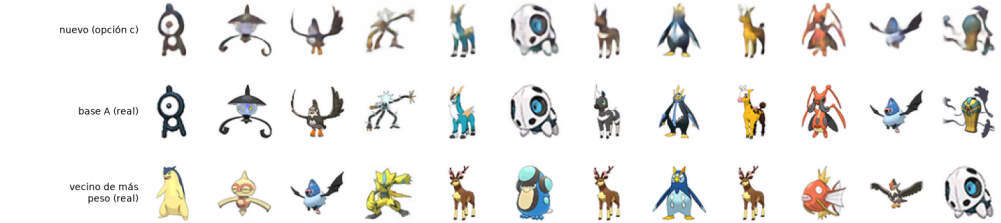
Qué muestra

- **A+B (vecinos):** criaturas con cuerpo, patas y paleta coherente. 13 de 14 superan el umbral de copia.
- **A+C (al azar):** vuelven a ser manchas.
- **A + ruido del posterior:** casi una copia de A (1 de 14 supera el umbral).

Lectura

El decodificador dibuja bien cerca de los códigos que vio. Más cómputo no agrega detalle: con 728 Pokémon distintos el límite son los datos y la verosimilitud por píxel.

Ej. 2c: cerca de los datos sale un Pokémon nuevo con contorno y ojos





Elegida a ojo entre las 12 más nítidas; entre paréntesis, el peso de cada Pokémon en la mezcla.

Modelo del Ej. 2b sin reentrenar: $z = w_A \mu_A + \sum_j w_j \mu_{B_j}$, con A un Pokémon y B_j sus 3 vecinos latentes.

- **2022 de 3000** mezclas no son copias (mismo umbral del Ej. 2).
- Nitidez de las 12 elegidas: **0,20**, contra 0,11 del prior y 0,25 de un sprite real.
- La elegida está a 13,4 del dato más cercano (umbral 11,1).

Límite: es una base deformada hacia sus vecinos, no una especie inventada desde cero.

Conclusiones

1. **Dos números alcanzan para 32 letras, pero hay que encontrarlos**: dos capas ocultas y Adam lo logran en 5/5 semillas (0 px); PCA con 2 componentes deja hasta 15 px.
2. **El DAE sólo limpia si se entrena con ruido**: un AE sin ruido empeora la entrada; con d=8 y p_{train}=0,1 deja 0,44 px donde había 3,5 (87 % menos).
3. **El espacio latente de un autoencoder tiene huecos**: reconstruye e interpola entre códigos conocidos, pero no se puede muestrear.
4. **El término KL es lo que lo vuelve muestreable**: misma red y mismos datos, Fréchet 6,8 $\pm$ 0,6 (VAE) contra 19,1 (AE), con piso 4,1. El precio: 815 contra 802 nats de reconstrucción. β=1 es el equilibrio.
5. **Interpolar no prueba que un modelo sea generativo; muestrear del prior sí**: entre dos códigos de entrenamiento el AE también interpola bien.
6. **Parte de la borrosidad era de la MLP, no del VAE** (Ej. 2b): con convoluciones el Fréchet baja de 6,8 a 5,2 y desaparece el moteado; el detalle fino no aparece muestreando del prior ni con 4 h de GPU, pero sí generando cerca de los datos (Ej. 2c).

Relación entre los ejercicios

El Ej. 1 muestra qué puede hacer la compresión y dónde falla (un espacio latente sin estructura); el Ej. 2 agrega la estructura estadística que faltaba, reutilizando las mismas capas, el mismo optimizador y la misma pérdida de reconstrucción.

Limitaciones, próximos pasos y reproducibilidad

Limitaciones

- Ej. 1: sólo 32 patrones, sin validación (el enunciado pide representar el conjunto); un único tipo de ruido.
- Ej. 2 y 2b: muestras **sin detalle fino** (verosimilitud factorizada por píxel; 728 Pokémon, uno de cada uno). El modelo de 64×64 tiene una sola semilla.
- Fréchet sobre 20 componentes principales: sustituto grueso del FID. El filtro de selección es una heurística.
- 3 semillas por celda y una sola partición de validación.

Próximos pasos

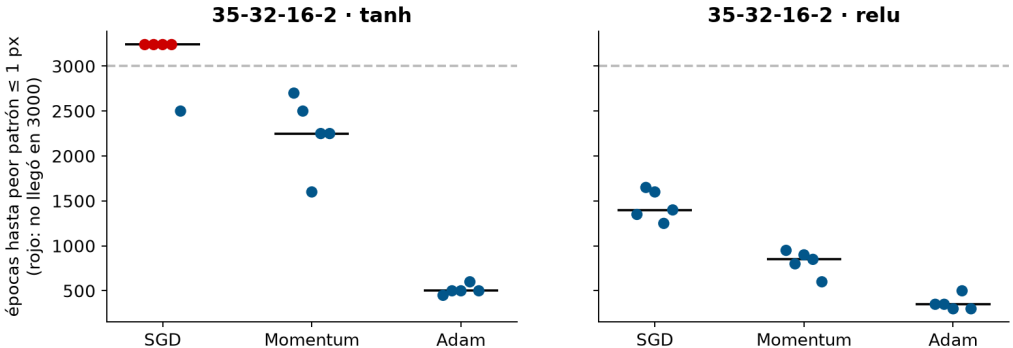
Más sprites por Pokémon; pérdida perceptual o adversaria.

Reproducibilidad

`github.com/yakdudi12/DIA02A` (carpeta `TP5`): notebooks, módulos, `README.md` y `config.json`. Semillas: 0–4 (Ej. 1), 0–2 (Ej. 2); modelos finales con semilla 0.

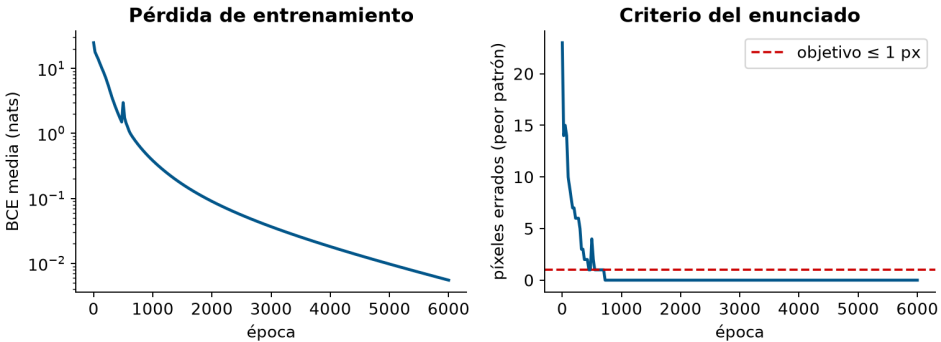
Bibliografía: apunte de la cátedra (*Artificializando*, cap. 18); Goodfellow et al. (2016), cap. 14; Aggarwal (2023), §3.4 y §5.10; Vincent et al. (2008); Kingma & Welling (2014, 2019); Doersch (2016); Higgins et al. (2017); Heusel et al. (2017).

Anexo A: épocas hasta ≤ 1 píxel según el optimizador (35–32–16–2)



Cada punto es una semilla; línea negra = mediana. Rojo = no alcanzó $\leq$1 píxel en 3000 épocas.

Anexo B: modelo final del Ej. 1a, curvas de entrenamiento

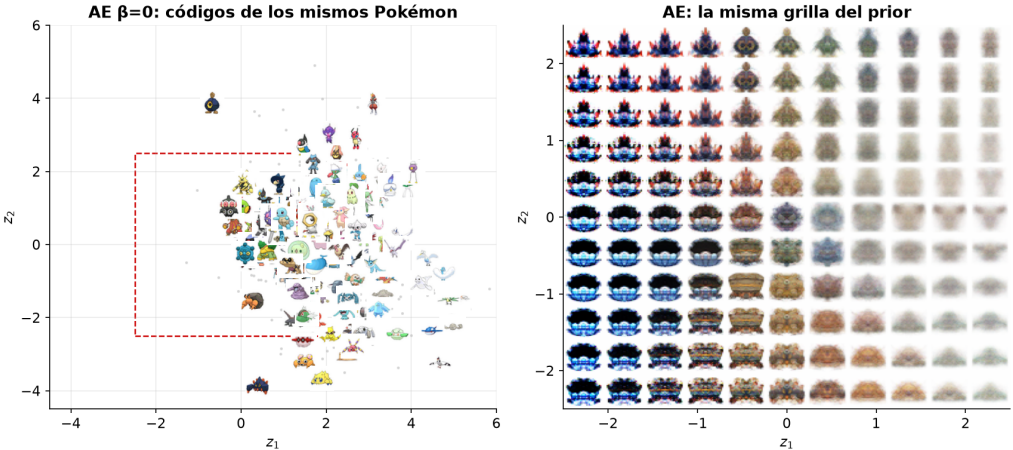


Anexo C: interpolación o $\rightarrow$ c en el espacio latente del autoencoder (Ej. 1a)



Distancia de Hamming de cada imagen al conjunto de entrenamiento: 0, 0, 2, 5, 6, 4, 1, 0, 0.

Anexo D: sin KL ($\beta = 0$, $d = 2$) más códigos quedan fuera del prior y hay regiones dominadas por un solo dato



Mismo modelo y datos con $\beta = 0$. El 74 % de los códigos cae en $[-2{,}5;\ 2{,}5]^2$ (VAE: 92 %). Fréchet 109 contra 46 del VAE (semilla 0).

Anexo E: reconstrucción de Pokémon que el modelo nunca vio



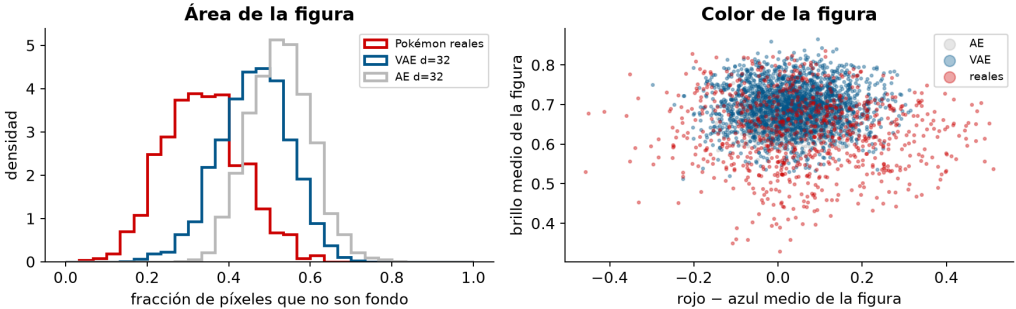
Con d=32 se recuperan silueta y paleta, no los detalles. Con d=2 queda una mancha del color dominante.

Anexo F: las interpolaciones son suaves también en el autoencoder



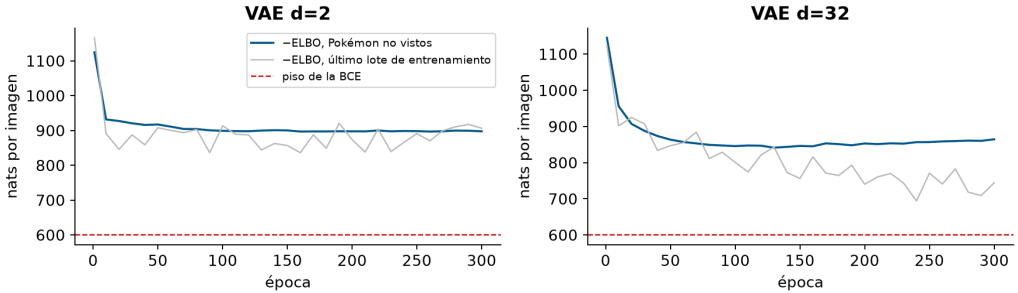
Interpolación lineal entre las medias μ de dos Pokémon (d=32). No distingue los modelos: la diferencia aparece al muestrear del prior.

Anexo G: área y color de las figuras generadas frente a los sprites reales



2000 muestras por modelo (d=32). Área media: reales 0,34; VAE 0,46; AE 0,53. El VAE promedia paletas: figuras menos saturadas.

Anexo H: curvas de entrenamiento del VAE ($\beta = 1$, semilla 0)



—ELBO por imagen en Pokémon no vistos (azul) y en el último lote de entrenamiento (gris).